ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Introducing Types and Typed Systems

In the previous lectures we have worked entirely within the pure lambda calculus — a universe in which there is only one kind of value: functions. Numbers, booleans, pairs, and data structures of every sort are all *encoded* as functions, and the only computation mechanism is β-reduction.

That uniformity is elegant, but it comes at a cost. Consider the applied lambda calculus we built on top of the pure one, with Church numerals and Church booleans. Nothing in the calculus itself prevents us from writing a term like

$$\mathsf{if}\ 5\ \mathsf{then}\ 0\ \mathsf{else}\ 1$$

What should this mean? The guard of a conditional is supposed to be a boolean — true or false — yet here we have a number. Since if is encoded as application in the lambda calculus, the expression does reduce — but it does not select either branch as intended. Instead, it produces a meaningless result. The problem is that the calculus treats true and 5 as the same kind of thing (they are both λ-terms), even though we, the programmers, intend them to play very different roles.

This is not a peculiarity of the lambda calculus. The same pattern arises wherever a single uniform representation is used for everything. In computer memory, for instance, every piece of data — characters, integers, pointers, instructions — is ultimately just a bit string of fixed size. The hardware does not know whether a given word represents the integer 65 or the character `A`; it is the *programmer* (or the compiler) that must keep track of these distinctions and enforce the appropriate discipline.

As Cardelli and Wegner observed, "*the road from untyped to typed universes has been followed many times in many different fields, and largely for the same reasons.*" Untyped universes are simple and general, but as soon as we start working in them — writing real programs, proving real theorems — we find ourselves distinguishing among different classes of values and insisting that operations be applied only to the "right" kind of argument.

Typed domains

In other words, untyped domains naturally decompose into **subsets with uniform behavior**, which we may take as our working definition of **types**. A type may be viewed as a set of values dressed with clothing (or a suit of armor) that:

- provides a protective covering that hides the values' underlying representation, and
- constrains the way values may be operated upon by operators and functions.

For example, integers are presented as atomic values with a transparent representation (in languages that leave bitwise operators aside...) and exhibit uniform behavior by having the same set of applicable operations. Similarly, functions from integers to integers have a protected, internal representation and behave uniformly in that they apply to arguments of a given type and produce results of a given type.

Having identified the intended ways that values of a certain type should be operated upon, we need tools to ensure that those intentions are actually complied with. Type systems provide one such tool: their main purpose is to impose constraints which prevent expressions from engaging in inconsistent interactions with other expressions.

To get a vivid sense of what "inconsistent interaction" looks like in practice, have a look at this video!

The code snippets shown in the video are funny (and extreme), but they are conceptually no different from the $\mathsf{if}\ 5\ \mathsf{then}\ 0\ \mathsf{else}\ 1$ example above: an operation is applied to values it was never meant to handle, and something meaningless — or surprising — happens. The goal of a type system is to rule out such situations *statically*, by inspecting only the structure of expressions, before any evaluation takes place.

The basic components of a type system may be described as follows:

1. *Type binding*: the mechanism by which we associate a type with each of the basic components of a program, such as constants and identifiers.
2. *Type checking/inference*: the process by which each compound expression gets associated with a type, ensuring that operators and functions are applied in a type-consistent way.

Typed Arithmetic Expressions

We now exemplify the concepts discussed so far and introduce the basic machinery of type systems by looking at a minimal language of boolean and arithmetic expressions. The language and its type system are nearly trivial, but precisely because of their simplicity they prove effective at illustrating the key techniques at work.

Syntax. The expressions of our simple language are defined by the following productions:

$$
\begin{array}{rcll}
a, b, c \in \mathrm{Const} & ::= & m, n, p & \text{numeric literals } 0, 1, 2, \ldots \\
& \mid & \mathsf{true} \mid \mathsf{false} & \text{boolean literals} \\
M, N, P \in \mathrm{Exp} & ::= & a, b, c & \text{constants(values)} \\
& \mid & M + N & \text{addition} \\
& \mid & M = N & \text{equality test} \\
& \mid & \mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P & \text{conditional}
\end{array}
$$

Note: The set of legal expressions may equivalently be defined by a system of inductive rules for $M \in \mathrm{Exp}$.

Evaluation. As we have shown in our previous lecture, all the expressions of this simple language (with the exception of the equality test) can be encoded as λ-terms. However, for the purpose of demonstrating the role of types and type systems, we build the syntactic forms from primitive constants and operators.

Evaluation of the expressions in our calculus is defined, as in the lambda calculus, as a syntactic transformation. However, instead of β-reduction, here we introduce a few ad-hoc rules to simplify arithmetic and boolean operators. The resulting reduction is still denoted $\to$ and defined by the following *δ-rules*.

Numeric Reductions

$$
\frac{(m \text{ and } n \text{ numeric literals})}{n + m \to n +_{\mathbb{N}} m}\ (\mathrm{add}_{+}) \qquad \frac{M \to M'}{M + m \to M' + m}\ (\text{add-L}) \qquad \frac{M \to M'}{m + M \to m + M'}\ (\text{add-R})
$$

The first rule defines the reduction for the numeric *redex*, i.e., an addition whose operands are both numeric constants. The notation $+_{\mathbb{N}}$ emphasises that the result is the arithmetic sum of the two numerals. The two additional rules are *congruence rules*: they define how to reduce compound expressions by reducing the enclosed redexes, proceeding left to right.

Boolean Reductions

$$
\frac{(a \text{ and } b \text{ different values})}{a = b \to \mathsf{false}}\ (\mathrm{eq}_{\neq}) \qquad \frac{}{a = a \to \mathsf{true}}\ (\mathrm{eq}_{=}) \qquad \frac{M \to M'}{M = N \to M' = N}\ (\text{eq-L}) \qquad \frac{M \to M'}{a = M \to a = M'}\ (\text{eq-R})
$$

$$\frac{}{\textsf{if true then } M \textsf{ else } N \to M}\ (\text{if}_t) \qquad \frac{}{\textsf{if false then } M \textsf{ else } N \to N}\ (\text{if}_f)$$

$$\frac{M \to M'}{\textsf{if } M \textsf{ then } N \textsf{ else } P \to \textsf{if } M' \textsf{ then } N \textsf{ else } P}\ (\text{if-guard})$$

The rationale is the same as for the numeric operators. We have four axioms defining the reductions for the corresponding redexes, and congruence rules to reduce a complex expression by reducing the enclosed redex.

Values, stuck terms, and errors

Introducing constants and operators as primitives with ad-hoc reduction rules brings about a new question concerning reduction and normal forms.

In the pure lambda calculus we defined a normal form as an expression without redexes. Since all closed expressions could be thought of as belonging to the same category — that of functions — normal forms served naturally as the notion of value.

In our present calculus, we could take the same approach and define a value as an expression without redexes, i.e., as a normal form of the reduction relation $\to$.

However, there are normal forms that don't look like values at all. Take for instance the terms $4 + \textsf{true}$ and $\textsf{if } 5 \textsf{ then } M \textsf{ else } N$: both are in normal form (no rule applies), yet they can hardly be considered values. Rather, they look like the representation of something that went *wrong* in the computation.

To distinguish normal forms that genuinely qualify as values from those that signal errors, we need a refined notion of value.

Definition — Values and Stuck Terms. The *values* of our calculus of arithmetic expressions are the numeric and boolean constants. A term that is not a value and that contains no redex is a *stuck term*.

Inherent in our interpretation of stuck terms as computational errors is the understanding that operators should be applied to their arguments with a certain discipline, and that when that discipline is not complied with we have an error.

Now, we would like to be able to tell — *without actually evaluating* a term — that its evaluation will not get stuck. To do that, we need to be able to distinguish between terms whose result will be a numeric value and terms whose result will be a boolean. That is the task of the type system.

We introduce two types, $\textsf{Nat}$ and $\textsf{Bool}$, for classifying terms in this way. The metavariables S, T, U, etc. range over types. Saying that "a term M has type T" means that we anticipate — *statically* — that M will evaluate to a value of the appropriate form.

The Typing Relation

The typing relation is formalized by means of *typing judgements* of the form $M : T$ (read "M has type T"), and defined inductively by the following system of rules:

$$\frac{}{\textsf{true} : \textsf{Bool}}\ (\text{T-True}) \qquad \frac{}{\textsf{false} : \textsf{Bool}}\ (\text{T-False}) \qquad \frac{}{n : \textsf{Nat}}\ (\text{T-Nat})$$

$$\frac{M : \textsf{Nat} \quad N : \textsf{Nat}}{M + N : \textsf{Nat}}\ (\text{T-Add}) \qquad \frac{M : T \quad N : T}{M = N : \textsf{Bool}}\ (\text{T-Eq}) \qquad \frac{M : \textsf{Bool} \quad N : T \quad P : T}{\textsf{if } M \textsf{ then } N \textsf{ else } P : T}\ (\text{T-If})$$

Let us read these rules carefully:

- Rules (T-True), (T-False), and (T-Nat) are *axioms* (rules with empty premises): they assign the type Bool to the boolean constants true and false, and the type Nat to every numeral n.
- Rule (T-Add) assigns Nat to an expression $M + N$ provided both M and N have type Nat.
- Rule (T-Eq) assigns Bool to $M = N$ provided M and N share the same type T. The two occurrences of the same metavariable T express precisely this constraint: the types of M and N must coincide, but are not fixed to any particular type.
- Rule (T-If) assigns type T to $\mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P$ provided the guard M has type Bool and the two branches N and P both have type T. Again, the two uses of T constrain the branches to have the same type.

Type Safety: *well-typed terms don't go wrong*

The fundamental property of a type system is *safety* (also called *soundness*), summarized by the slogan: **well-typed terms do not go wrong**.

We have already chosen how to formalize what it means for a term to "go wrong": it means reaching a *stuck term*. What we want to know, then, is that well-typed terms never get stuck. This is established in two steps, commonly known as the *progress* and *preservation* theorems.

Theorem (Progress). If $M : T$ then either M is a value, or there exists N such that $M \to N$.

Theorem (Preservation). If $M : T$ and $M \to N$, then $N : T$.

The Progress theorem tells us that well-typed terms are not stuck: they are either already values, or they can take a reduction step. The Preservation theorem (also known as *Subject Reduction*) guarantees that if a well-typed term takes a step, the resulting term is also well-typed — and at the same type.

Together, progress and preservation ensure that every term whose evaluation would reach a stuck state is ruled out by the type system as *ill-typed*.

A subtlety. This does *not* mean that every ill-typed term will necessarily get stuck. For instance, $\mathsf{if\ true\ then\ 0\ else\ false}$ is ill-typed (the branches have different types), but its evaluation reduces to $\mathsf{0}$, which is a perfectly good value.

Being well-typed is therefore a *sufficient* but not *necessary* condition for the absence of stuck states. This should not be surprising: the set of terms whose evaluation does not get stuck is not decidable in general (by Rice's Theorem), so no decidable analysis can characterize it exactly. The purpose of a sound type system is to enforce a decidable approximation — one that satisfies progress and preservation — without being too conservative, i.e., it should accept most of the programs we actually care about writing.

Proofs of the main type-system properties

We now examine the main properties of our type system. We begin with three structural lemmas about typing derivations: these are relatively straightforward consequences of the shape of the rules, but they are essential building blocks for the safety theorems that follow.

Lemma (Inversion of the Typing Relation).

- If $\mathsf{true} : T$ then $T = \mathsf{Bool}$.
- If $\mathsf{false} : T$ then $T = \mathsf{Bool}$.
- If $\mathsf{if}\ M\ \mathsf{then}\ N\ \mathsf{else}\ P : T$ then $M : \mathsf{Bool}$, $N : T$ and $P : T$.
- If $n : T$ then $T = \mathsf{Nat}$.
- If $M + N : T$ then $T = \mathsf{Nat}$ and $M : \mathsf{Nat}$ and $N : \mathsf{Nat}$.

- If $M = N : T$ then $T = \mathsf{Bool}$ and $M : S$ and $N : S$ for some type S.

Proof sketch. For each syntactic form there is exactly one typing rule that could derive a judgement for it. The clauses follow by inspection.

The Inversion Lemma is sometimes called the *Generation Lemma*: given a valid typing statement, it shows how a proof of that statement could have been generated. Crucially, it also provides us with a *type-checking algorithm*: for each term, it tells us how to compute its type (if it has one) from the types of its sub-terms.

Theorem (Uniqueness of Types).
For each term M there exists at most one type T such that $M : T$. Moreover, there is exactly one derivation of the typing judgement from the inference rules.

Proof sketch. By induction on the structure of M, applying the appropriate clause of the Inversion Lemma at each step. The key observation is that each syntactic form determines a unique rule, and the Inversion Lemma pins down the types of the sub-terms, to which the induction hypothesis applies. (You are asked to work out the details in Exercise 4 below.)

Lemma (Canonical Forms).

- If M is a value and $M : \mathsf{Bool}$, then M is either true or false.
- If M is a value and $M : \mathsf{Nat}$, then M is a numeric literal n.

Proof sketch. Values have three possible forms: true, false, or a numeric literal. By uniqueness of types, a value of type Bool cannot also have type Nat, so it must be true or false; dually for Nat.

These three lemmas are the scaffolding we need. Now we turn to the two theorems that constitute the safety guarantee: Progress and Preservation. These proofs are the first place where we see *rule induction* applied to a concrete system, putting to work the techniques from the previous lecture on inductive definitions.

Recall the principle of rule induction: to show that a property P holds for all elements of a set inductively defined by a judgement form J, it suffices to show that P *respects* all the rules defining J — that is:

- for each axiom, that P holds for the conclusions
- for each inference rule, that if P holds for all the premises (the inductive hypotheses), then P holds for the conclusion.

The typing relation $M : T$ is defined inductively by six rules: three axioms — (T-True), (T-False), (T-Nat) — and three inference rules — (T-Add), (T-Eq), (T-If). So to prove that a property holds for all well-typed terms, we must show it respects each of these six rules.

Proof of the Progress Theorem.

Let $P(M)$ be the property: "M is a value or there exists N such that $M \to N$." We show that $P(M)$ holds for all M such that $M : T$, by rule induction on the typing judgement $M : T$.

Respecting (T-True), (T-False), (T-Nat). In each case M is a constant, hence a value. So $P(M)$ holds immediately.

Respecting (T-Add). $\dfrac{\mathrm{M} : \mathsf{Nat} \qquad \mathrm{N} : \mathsf{Nat}}{\mathrm{M} + \mathrm{N} : \mathsf{Nat}}$

- By the inductive hypotheses, $P(M)$ and $P(N)$ both hold. We must show that $P(M + N)$ also holds.
- If M is not yet a value, then $P(M)$ implies that $M \to M'$ for some M', and then $M + N \to M' + N$ by (add-L).
- If M is a value (hence a numeral) but N is not, then $N \to N'$ for some N', and $M + N \to M + N'$ by (add-R).
- If both are values, both are numerals, and $M + N$ reduces by (add_+).

Respecting (T-Eq). $\dfrac{\mathrm{M} : \mathrm{T} \quad \mathrm{N} : \mathrm{T}}{\mathrm{M} = \mathrm{N} : \mathsf{Bool}}$

- If M and N are the same term, as in this case $M = N \to \mathsf{true}$. For all the other cases:
- By the inductive hypotheses, $P(M)$ and $P(N)$ both hold. We must show that $P(M = N)$ also holds.
- If M is not yet a value, then $P(M)$ implies that $M \to M'$ for some M', and then $M = N \to M' = N$ by (eq-L).
- If M is a value but N is not, then $N \to N'$ for some N', and $M = N \to M = N'$ by (eq-R).

Respecting (T-If). $\dfrac{M{:}\mathsf{Bool} \quad N_t{:}T \quad N_f{:}T}{\mathsf{if}\ M\ \mathsf{then}\ N_t\ \mathsf{else}\ N_f{:}T}$

- By the inductive hypothesis, $P(M)$ holds: either M is a value or $M' \to M'$ for some M'.
- If M' is a value, then by the Canonical Forms Lemma, it is either true or false, so the conditional reduces to N_t or to N_f.
- If $M \to M'$, then $\mathsf{if}\ M\ \mathsf{then}\ N_t\ \mathsf{else}\ N_f$ reduces to $\mathsf{if}\ M'\ \mathsf{then}\ N_t\ \mathsf{else}\ N_f$ by (if-guard).

In both sub-cases the term can take a step, so P is respected. □

Proof of the Preservation Theorem.

Let $P(M, T)$ be the property: "for all N, if $M \to N$ then $N : T$." We show that $P(M, T)$ holds for all M, T such that $M : T$, by rule induction on the typing judgement $M : T$.

Respecting (T-True), (T-False), (T-Nat). Constants are values and cannot step, so P holds vacuously.

Respecting (T-Add). $\dfrac{\mathrm{M} : \mathsf{Nat} \quad \mathrm{N} : \mathsf{Nat}}{\mathrm{M} + \mathrm{N} : \mathsf{Nat}}$

- By the induction hypothesis $P(M, \mathsf{Nat})$ and $P(N, \mathsf{Nat})$ both hold.
- Suppose $M + N \to R$. There are three sub-cases, corresponding to the three reduction rules for addition:
  - ($\mathrm{add}_{=}$): both M and N are numerals, and R is their arithmetic sum. Then R is itself a numeral, so $R : \mathsf{Nat}$ by (T-Nat).
  - (add-L): $M \to M'$ and $R = M' + N$. By the inductive hypothesis, $M' : \mathsf{Nat}$. Together with $N : \mathsf{Nat}$, rule (T-Add) gives $R : \mathsf{Nat}$.
  - (add-R): $N \to N'$ and $R = M + N'$. Symmetric to the previous case.

Respecting (T-Eq). $\dfrac{\mathrm{M} : \mathrm{T} \quad \mathrm{N} : \mathrm{T}}{\mathrm{M} = \mathrm{N} : \mathsf{Bool}}$.

- By the induction hypothesis $P(M, T)$ and $P(N, T)$ both hold.
- Suppose $M = N \to R$: There are the following subcases
  - ($\mathrm{eq}_{=}$): in this case R is true and $R : T$ by (T-True)
  - ($\mathrm{eq}_{\neq}$): in this case R is false and $R : T$ by (T-False)
  - (eq-L): $M \to M'$ and R is $M' = N$. By the induction hypothesis $M' : T$ and then $R : T$ by $N : T$ and $T - Eq$
  - (eq-R): symmetric to the previous case.

Respecting (T-If). $\dfrac{M{:}\mathsf{Bool} \quad N_t{:}T \quad N_f{:}T}{\mathsf{if}\ M\ \mathsf{then}\ N_t\ \mathsf{else}\ N_f{:}T}$

- By the induction hypothesis $P(M, \mathsf{Bool})$, $P(N_t, T)$ and $P(N_f, T)$ all hold.
- Suppose $\mathsf{if}\ M\ \mathsf{then}\ N_1\ \mathsf{else}\ N_2 \to R$. There are three sub-cases:
  - (if_t): $M = \mathsf{true}$ and $R = N_t$. Then $R : T$ follows directly from the premise $N_t : T$.
  - (if_f): $M = \mathsf{false}$ and $R = N_f$. Then $R : T$ follows directly from the premise $N_f : T$.

- (if-cong): $M \to M'$ and $R = \mathsf{if}\ M'\ \mathsf{then}\ N_t\ \mathsf{else}\ N_f$. By the inductive hypothesis $P(M, \mathsf{Bool})$, we obtain $M' : \mathsf{Bool}$. From this, together with $N_t : T$ and $N_f : T$, we apply (T-If) to derive $R : T$. □

Exercises

1. Subject expansion? We have established subject reduction (preservation). It is natural to wonder whether the converse — *subject expansion* — also holds. Is it always the case that if $M \to N$ and $N : T$ then $M : T$? If so, prove it. If not, give a counterexample.

2. Typing of conditionals. As argued above, ill-typed terms such as $\mathsf{if\ true\ then\ 0\ else\ false}$ may actually behave well during evaluation. Show that if we modify the typing rule for conditionals to make this term well-typed, we can construct a well-typed term that gets stuck.

3. Typing of equality. In our typing rule for $M = N$, we require that M and N have the same type.

- How would you modify the rule to relax this constraint (admitting the equality test on terms of different types)?
- Which properties of the original system would change under the new rule?
- In particular, would the new system still be sound? If so, prove it. Otherwise, provide a counterexample.

4. Uniqueness of types (proof). Prove the Uniqueness of Types theorem by induction on the structure of M. For each syntactic form, use the appropriate clause of the Inversion Lemma to determine the types of the sub-terms, then apply the induction hypothesis. (*Hint:* the conditional case is the most instructive — start there.)

Credits

The discussion of untyped universes and the definition of types as "subsets with uniform behavior" draws on Luca Cardelli and Peter Wegner, *On Understanding Types, Data Abstraction, and Polymorphism*. Computing Surveys, 17(4):471–522, December 1985.

The section on arithmetic expressions is adapted from Chapter 8 of Benjamin C. Pierce, *Types and Programming Languages*. MIT Press, 2002.